



infgotoinf

10 часов назад

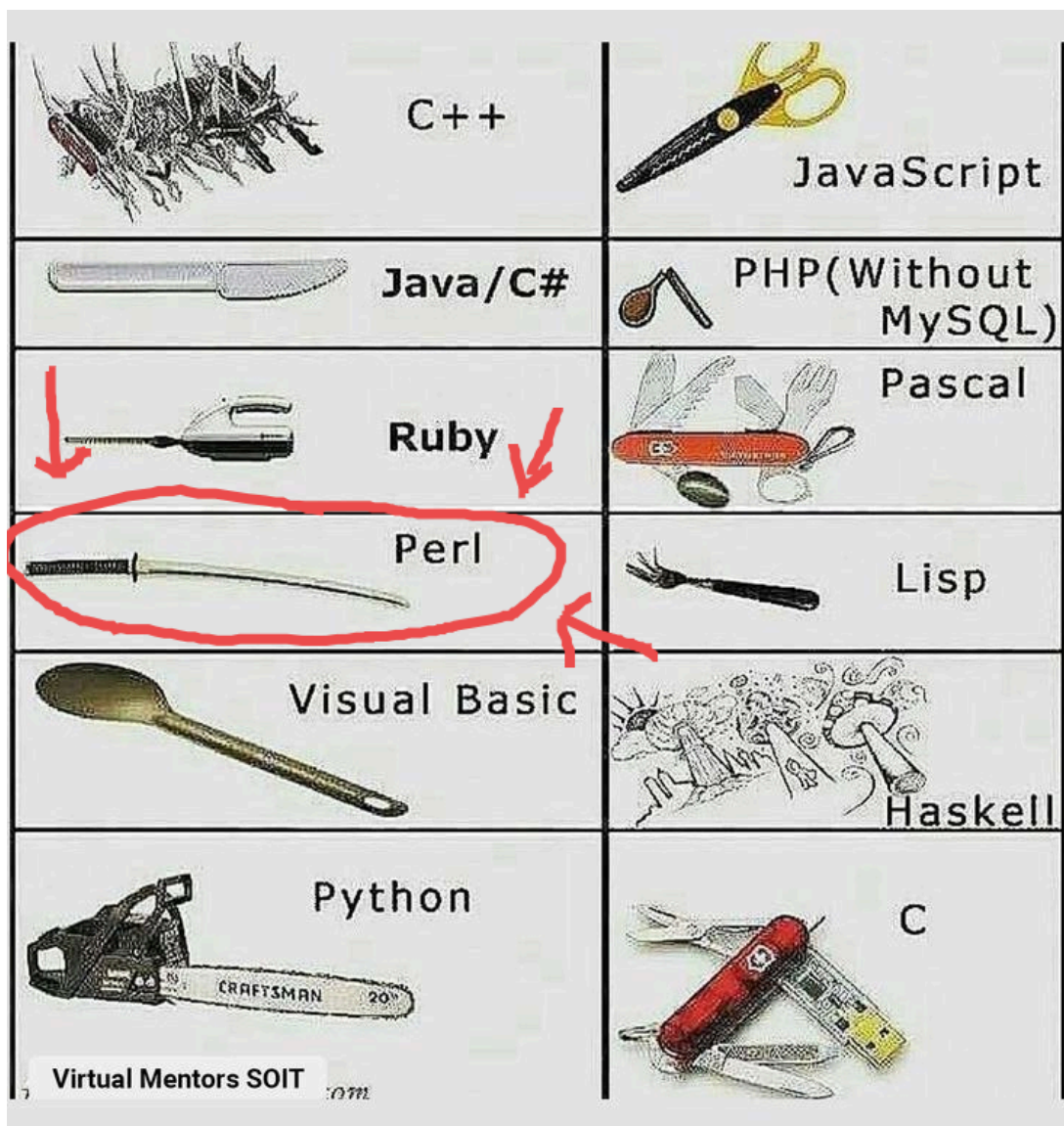
## Perl — зря забытый язык программирования?

11 мин 8.4К

Perl\*, Программирование\*

Из песочницы

В настоящее время Perl обделяется вниманием: о нём мало что и где можно услышать и увидеть. При этом Perl действительно уникальный язык программирования, который может предложить что-то новое, и особенности которого сильно выделяют его среди других. И сегодня я вам о нём поведаю, а также расскажу о его фичах с примерами его применения.



## Что из себя представляет Perl?

Perl — это скриптовый язык (как Bash или Python), разработанный ещё в 1987 году Лэри Воллом. Perl — динамически типизированный. По синтаксису код на нём выглядит как что-то между Python, C, и чем-то своим, при этом одним из его девизов является — «Здесь больше одного способа это сделать», что отражается в исключительной гибкости языка.

Пример вывода “Hello, World!”:

```
my $var = "World"; # Инициализация локальной переменной $var

print ("Hello, $var!\n");      # Стандартный пример вывода "Hello, World!"
print "Hello, $var!\n";      # Без скобочек
print "Hello, ", $var, "!\n";  # В виде листа аргументов, разделённых запятой
print "Hello, " . $var . "!\n"; # В виде строки, которая соединяется в одну точками
printf "Hello, %s!\n", $var;   # printf
say "Hello, $var!";           # say - тоже самое, что и print, только с переносом строки на конце
                              # Есть в Perl 5.10 и позднее
```

Объяснить с  SourceCraft

Пример отсчёта от 10 до 1:

```
# Стандартный для C for loop
for (my $i = 10; $i > 0; --$i) {
    say $i;
}

# foreach по перевёрнутому массиву от 1 до 10
foreach my $i (reverse 1..10) {
    say $i;
}

# Никакой разницы между foreach и for нет
for my $i (reverse 1..10) {
    say $i;
}

# Запись и вывод из дефолтного значения $_
for (reverse 1..10) {
    # То же самое как если бы мы записали
    # for my $_ (reverse 1..10)
    say $_;
}

# В одну строку
say $_ for reverse 1..10;
```

Объяснить с 

## Кроссплатформенность и простота работы с системой.

Так как Perl изначально создавался как скриптовый язык общего назначения для Unix, на многих Unix системах он стоит по умолчанию. Можете сами проверить есть ли он у вас `whereis perl`.

Конечно для пользователей прекрасной и неповторимой Windows не всё так просто, но в отличие от Bash, не нужны никакие танцы с бубном для эмуляции Linux и достаточно просто [скачать интерпретатор Perl](#).

Perl позволяет очень просто работать с файловой системой: читать, изменять, удалять, файлы и папки, свой простой аналог функции `cat` или `sort` в Perl можно написать всего в одну строку:

```
# cat в одну строку, который принимает в себя любое число файлов и выводит их содержани
# Даже точка с запятой на конце не нужны потому, что в конце блока её можно не ставить.

print <> # Алмазный оператор <> - это оператор ввода пользовательского выбора

# Этот оператор работает следующим образом: если пользователь передал в программу файлы
# то он возвращает их содержание, если нет, то принимает ввод с консоли до тех пор пока
# пользователь не вернёт end-of-file код (Ctrl-D) и возвращает введённое
```

[Объяснить с](#) 

```
print sort <> # Сортирует все строки файла и выводит их в консоль
```

[Объяснить с](#) 

```
# Аналог 'nl -b a' который нумерует все строки файла и выводит их в консоль
```

```
printf "%6d $_", $_. while <> # $_. возвращает номер строки
```

[Объяснить с](#) 

► [Что-то подобное алмазному оператору <> на языке Perl можно написать следующим образом:](#)

Можете сами попробовать создать [main.pl](#) вставить туда код и запустить:

```
perl main.pl PATH_TO_FILE
```

[Объяснить с](#) 

В Perl также просто можно вызывать системные команды и обрабатывать их вывод, что в сочетании с непревзойдённой работой с текстом (о которой будет следующий параграф) может сильно упростить жизнь.

```
# Оператор `` исполняет команду и возвращает результат её вывода, когда она завершится
my $ping = `ping google.com -w 2`;
print $ping
```

[Объяснить с](#) 

```
# -| открывает программу на чтение вывода
open( my $ping, '-|', 'ping google.com -w 2' );
```

```
# Будет выводить в консоль результат вывода команды по ходу её выполнения
```

```
while my $line (<$ping>) {  
    print $line  
}
```

Объяснить с 

Таким образом я считаю, что Perl куда проще, мощнее и гибче чем Bash, но конечно наверное большинство знает Bash, да и его использование более распространено. Но тем не менее, если есть возможность писать скрипт на Perl, вместо того, чтобы делать это на Bash, то почему бы и нет, особенно если вы хотите, чтобы этот скрипт работал не только на Unix, но и на Windows, хотя возможно для такого бы также хорошо подошёл Python. Всё зависит от ваших целей, желаний и ограничений.

## Работа с текстом

Работа с текстом - это пожалуй самая сильная сторона Perl и главный фактор послуживший его популярности, ну и причина почему я сам начал его изучать. Из-за глубокой интеграции с regex и тем как страшно и непонятно код на regex может выглядеть, иногда можно услышать, что о Perl отзываются как «write only language», но это глубоко не так и даже наоборот и сейчас я докажу это вам на своём примере.

Так вот, когда я делал 3D игру с raylib, у меня появилась потребность в использовании 3D редактора карт и я нашёл [TrenchBroom](#). Для энтити, которых можно добавлять на карту TrenchBroom использует формат .FGD, а этот формат очень специфичный, неприятный и с ним довольно таки неудобно работать, ну и мне в любом случае придётся этих энтити прописывать на C/C++, поэтому зачем вообще к этому формату прикасаться? Вместо этого можно написать программу, которая принимает в себя C/C++ код в виде классов/структур и переводит его в .FGD.

То есть нужна программа которая переводит файл такого формата:

```
#ifndef EXAMPLE_H  
#define EXAMPLE_H  
  
// Example class  
struct Example {  
    const char* name;  
    int hp; // Character's health  
    void spawn();  
    // Subtracts damage from hp  
    void takeDamage(int damage);  
};  
  
struct Example2 {  
    int ammount = 0; // Ammount of stuff  
    Example2();  
    // Returns true if ammount is even  
    bool isEven();  
};  
  
#endif //EXAMPLE_H
```

Объяснить с 

Вот в это:

```
@PointClass size(-16 -16 -32, 16 16 32) = Example : "Example class"
[
    name(string): : : ""
    hp(integer): : : "Character's health"
]
@PointClass size(-16 -16 -32, 16 16 32) = Example2 : ""
[
    ammount(integer): : 0 : "Ammount of stuff"
]
```

Объяснить с 

Ну для начала определим как оно вообще должно из первого сделать второе:

1. Структуры типа `// COMMENT\n struct NAME {...};` переводятся в .FGD классы формата `@PointClass size(-16 -16 -32, 16 16 32) = NAME : "COMMENT" [...]` при том, что комментарии опциональны.
2. Поля структуры типа `TYPE NAME = DEFINITION; // COMMENT` переводятся в поля .FGD класса формата `NAME (TYPE) : : DEFINITION : "COMMENT"`, при этом определения полей и комментарии опциональны.
3. Типы `int` и `const char*` переводятся в `integer` и `string`.
4. Все лишние методы класса, лишние комментарии, макросы и инклюды полностью удаляются.

Перед тем как перейти к решению на Perl давайте посмотрим как можно добиться подобного результата на другом языке. Тут я уточню, что кроме C/C++ больше ничего то толком и не знаю, поэтому для меня вариантов не много. Очевидно, что для текущей задачи нужно использовать регулярные выражения, а то в противном случае решение получится чрезмерно большим, некрасивым и сложным.

`regex.h` простой и подойдёт для валидации небольшого набора данных, к примеру IP адрес, пароль, имя пользователя и тп., но если нужно что-то побольше, то тут его функционала уже начинает не хватать, и получаются очень длинные однострочные ~~write-only~~ `regex` перекспы. `std::regex` уже получше, но с ним всё та же проблема.

Есть [PCRE2 \(Perl Compatable Regular Expressions\)](#), который имеет очень мощный функционал, но который требует вызова функций с кучей параметров, что создаёт много шума из-за чего его труднее воспринимать, и для маленькой программки с одной единственной целью это уже перебор.

▸ [Пример кода из их репозитория](#)

А вот `boost/regex.hpp` - это пожалуй самый лучший вариант из всех, тк совмещает в себе простоту и мощь `regex` Пёрла, никак при этом не давя на глаза.

▸ [Пример того как первый шаг конвертации мог бы выглядеть с `boost/regex.hpp`](#)

У питона есть модуль `re` тоже способный на написание понятного `regex` кода:

```

charref = re.compile(r"""
    &[#]                # Start of a numeric entity reference
    (
        0[0-7]+         # Octal form
        | [0-9]+         # Decimal form
        | x[0-9a-fA-F]+  # Hexadecimal form
    )
    ;                    # Trailing semicolon
""", re.VERBOSE)

```

Объяснить с 

Ну а как бы этот конвертер выглядел на самом Perl? Как-то [так](#), но если вкратце и с пояснениями, то так:

```

#!/usr/bin/env perl

# Используем самую последнюю версию Perl она по деолту включает strict и warnings
use v5.42;
# Для корректной работы utf8
use utf8;
binmode STDOUT, ':encoding(UTF-8)';
binmode STDIN, ':encoding(UTF-8)';

# Первый аргумент - С структура, второй - путь куда будет экспортирован конвертированный
my $source = $ARGV[0];
my $dest = $ARGV[1];

# Чтение открытого файла
open (my $source_file, '<', $source)
    or die "Could not open source file: $!\n";

# Соединяем все строки в одну
my $source_code = join '', <$source_file>;

# Заменяем структуру на .FGD
$source_code =~ s{
    # Необязательный комментарий к структуре
    (? : ^\s* /\s* (?<struct_comment>[^\n]*) )?

    # Блок структуры
    (?<struct>
        \s* struct\s+ (?<struct_name>w+\d?)\s* \{
            (?<struct_content>.*?)
        \};
    )

} {@PointClass size(-16 -16 -32, 16 16 32) = ${struct_name} : "${struct_comment}"
[
    ${struct_content}
]

}mgxs; # m для того, чтобы $^R были для каждой строки, а не всего файла
# g для замены всех совпадений, а не только первого

```

```

# х для того, чтобы regex игнорировал пробельные символы и всё можно
# было читаемо разместить
# s для того, чтобы любой символ . также принимал в себя \n

# Замена полей структуры на поля .FGD класса
$source_code =~ s{
# Парс типа, имени, определения и комментария поля
\s* (?<variable_type>[\w\s*::<>]+?)\s+
    (?<var_name>\w+)
    (?:\s* =\s* (?<default_value>\w+?))??;
    (?:\s* /\s* (?<var_comment>[^\n]+))??
}{
    ${var_name} ${variable_type} : : ${default_value} : "${var_comment}"
}mgxs;

# Удаление всех лишних строк
$source_code =~ s{
^\s*
# Строки начинающиеся с #, //, заканчивающиеся с ; с возможным комментарием
# и просто пустые строки
( \#.+ | /\..*? | .*?; (\s*/\s*)? | )
\R
}{}mgx;

# Переименование типов данных
$source_code =~ s/\((const )?int\)/(integer)/g;
$source_code =~ s/\((const )?char*\)/(string)/g;

# Запись файла
open (my $dest_file, '>', $dest)
    or die "Could not create dest file: $!\n";

```

Объяснить с 

Как вы видите с Perl проще, чем в любом другом языке, писать сложные регулярные выражения, но стоит ли оно того?.. 🤔

## Пользовательские модули CPAN

Для Perl написано очень много пользовательских модулей для самых разных задач (прямо как и для Python), но пожалуй самыми полезными из них являются Getopt::Long и Pod::Usage, которые позволяют очень просто передавать параметры в программу и документировать код. Многие стандартные модули идут сразу вместе с Perl (включая те, про которые я только что рассказал), а те, что и не идут можно достаточно просто скачать. Для этого я рекомендую использовать [cpanm](#), тогда процесс установки любого модуля упрощается до `sudo cpanm MODULE`. На NixOS я просто пишу такой простой шелл конфиг:

► [shell.nix](#)

Ну и напоследок давайте я вам покажу как можно использовать всё, о чём я только что рассказал, в одном едином [скрипте](#), который берёт список прокси серверов и пингует через них сайты:

```
#!/usr/bin/env perl

use v5.42;
use utf8;
binmode STDOUT, ':encoding(UTF-8)';
binmode STDIN, ':encoding(UTF-8)';
# Подключаем пользовательские модули
use Furl;
use Getopt::Long;
use Pod::Usage;

# Задаём дефолтные значения
my $help = 0;
my $proxy_list =
'https://cdn.jsdelivr.net/gh/proxify/free-proxy-list@main/proxies/all/data.txt';
my $timeout = 1;

# Обрабатываем входные параметры
GetOptions( 'help|?' => \$help
            , 'link-to-proxy|p=s' => \$proxy_list
            , 'timeout|t=i' => \$timeout);
pod2usage(1) if $help;
pod2usage(2) unless $ARGV[0];

# Открываем сайт со списком прокси и записываем все прокси в массив @proxy_list
my $furl = Furl->new(timeout => 5);
my @proxy_list = split( /\n/, $furl->get($proxy_list)->content );

# Записываем аргументы переданные в программу в список сайтов, которые мы будем
# пинговать
my @link_list = @ARGV;

my %proxy_hash;
my $counter = @proxy_list;
foreach my $proxy (@proxy_list)
{
    say $counter--, '..'; # Отсчёт того сколько прокси осталось

    foreach my $link (@link_list)
    {
        # Делаем curl запрос
        open my $fh, '-|',
        "curl -s -x GET -o /dev/null --write-out '%{exitcode} %{time_total}' --proxy

        while (my $line = <$fh>)
        {
            # Проверяем что запрос удался и если да, то выводим результат и
            # записываем его в %proxy_hash
            if ($line =~ /^(?<exit_code>\d+) (?<time>.+)/
            and ${exit_code} == 0 && ${time} < $timeout)
            {
                say "${time} - $proxy";
                $proxy_hash{$proxy} .= ${time};
            }
        }
    }
}

```

```

    }
}

# Сортируем все прокси по времени и выводим
foreach (sort {$proxy_hash{$a} <=> $proxy_hash{$b}} keys %proxy_hash) {
    say "$proxy_hash{$_} - $_";
}

# perlpod документация
__END__

=head1 SYNOPSIS

main.pl [options...] <urls...>

=head1 OPTIONS

=over 4

=item B<-h, --help>

Prints this message.

=item B<-p, --link-to-proxy> <url>

Link to a site from that to fetch proxy server links.

Default is: https://cdn.jsdelivr.net/gh/proxify/free-proxy-list@main/proxies/all/data.

=item B<-t, --timeout> <seconds>

Max time to wait for a response before switching to next proxy.

Default is: 1

=back

=cut

```

Объяснить с 

▸ [Результат работы программы](#)

## Ресурсы для дальнейшего изучения

Если вас заинтересовал Perl, то я рекомендую следующие ресурсы к ознакомлению:

- [perldoc/perlintro](#) - краткое введение в Perl и его возможности.
- [perldoc/perl](#) - документация всего Perl.
- [perldoc/perlvar](#) - \$\_, \$., \$!, \$\$ и тд.
- [perldoc/perlre](#) - о регулярках в Perl.

- [PerlTutorial](#) - простые tutorиалы для начинающих.
- [pelmaven/perl-tutorial](#) - блог на разные темы, знать которые может пригодиться.
- [metacpan](#) - сайт с пользовательскими модулями для Perl.

Ну а также крайне особо сильно настоятельно рекомендую прочитать [Learning Perl 8th Edition](#), очень хорошая книга 2021-ого года. Если к тому времени, как вы читаете эту статью выйдет новое издание — читайте его. Удачи!

## Выводы

В процессе написания этой статьи я понял, что уникальность и особенность языка Perl, к сожалению не делает его незаменимым инструментом, а очень даже заменимым (заменимым на Python). У языка есть свои особенности, он по-своему необычен, интересен и крут, но есть ли сейчас много толка в его изучении? Ну насколько много, это вопрос хороший, но толк в этом для меня точно был.

**Теги:** язык программирования, perl, regex, обучение, regex, обзор языка

**Хабы:** [Perl](#), [Программирование](#)



### Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



5

Карма

[@infgotoinf](#)

Пользователь

Подписаться



### Хабр доступен 24/7 благодаря поддержке друзей

#### Хабр Курсы для всех

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

Комментарии 8

Публикации

ЛУЧШИЕ ЗА СУТКИ    ПОХОЖИЕ

**the\_bat**  
23 часа назад

### Ремонт блока питания с Power Delivery. 470 граммов электроники

 Простой     4 мин     16K

 +96     30     27

**interpres**  
18 часов назад

### Электроинструмент становится хуже, и это делается намеренно

 Простой     11 мин     26K

Обзор

Перевод

 +70     45     82

**prohronus**  
18 часов назад

### Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин     9.3K

Кейс

 +51     9     9

**alizar**  
22 часа назад

### Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой     7 мин     12K

Обзор

 +37     59     18

**TrexSelectel**  
19 часов назад

### Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин     8.7K

Обзор

 +27     10     0

**AlexSerbul**  
23 часа назад

### Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний     14 мин     9.9K

Мнение

+26

33

11



mariklolik

18 часов назад

## Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.4K

Кейс

+23

23

6



nick\_dyuba

22 часа назад

## Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіrіTіr

5 мин

7.1K

Recovery Mode

+22

8

5



Mimizavr

23 часа назад

## «Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин

6.9K

+17

9

7



infgotoinf

10 часов назад

## Perl — зря забытый язык программирования?

11 мин

8.4K

Из песочницы

+16

13

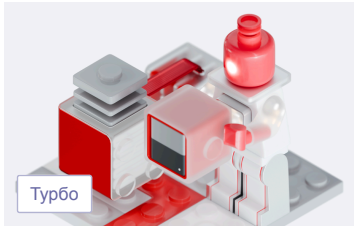
8

## Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

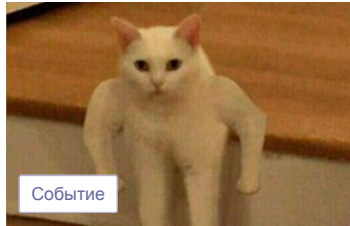
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и выиграй призы



Качаем к лету хард- и софт-скиллы на айтишных ивентах

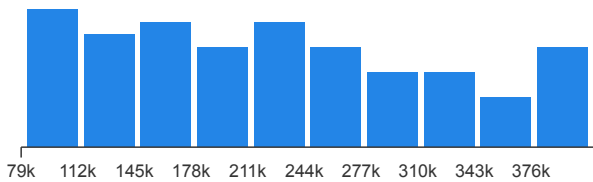


Удалёнка в фарме: большие данные и никакой магии

#### СРЕДНЯЯ ЗАРПЛАТА В IT

**221 623** ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 32 194 анкет, за 1-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!



[Проверить свою зарплату](#)



## ЧИТАЮТ СЕЙЧАС

---

Электроинструмент становится хуже, и это делается намеренно

 26K  82

---

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14

---

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.9K  15

---

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.4K  11

---

+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  10

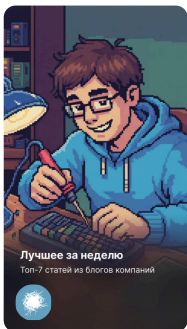
---

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо



## ИСТОРИИ



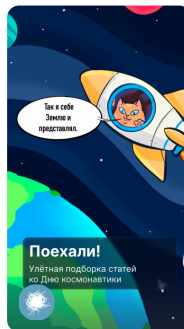
Годнота из блогов компаний



Только 10% решат эту головоломку



Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать



## БЛИЖАЙШИЕ СОБЫТИЯ

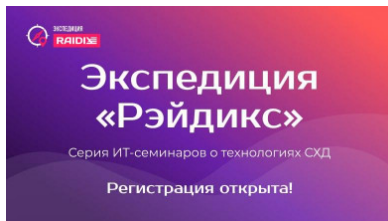


2 марта – 10 августа

**PROBA Awards —  
международная  
коммуникационная  
премия, учрежденная в  
2000 году**

Санкт-Петербург

Маркетинг



3 марта – 9 июня

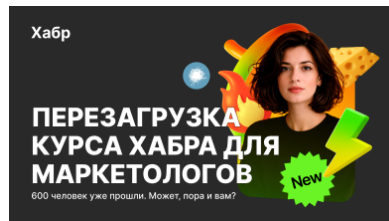
**Экспедиция «Рэйдикс»**

Екатеринбург • Новосибирск •  
Краснодар • Минск • Калининград •  
Ростов-на-Дону

Разработка

Другое

Больше событий в календаре



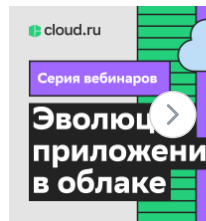
9 марта – 1 июня

**Всё, что нужно, чтобы  
стартовать продвижение  
бренда на Хабре**

Онлайн

Маркетинг

Больше событий в календаре



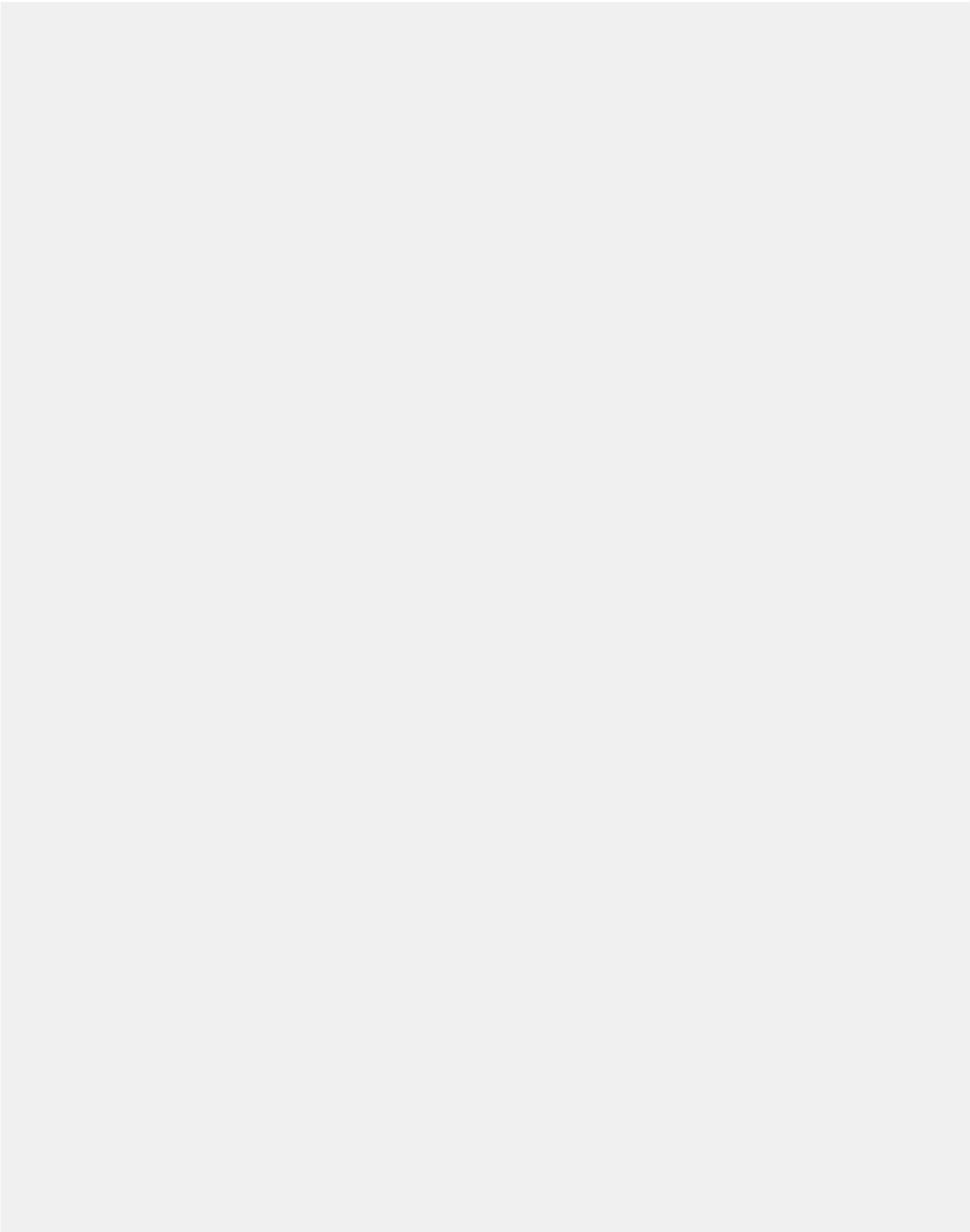
19 марта – 28 апреля

**Серия вебинаров  
«Эволюция приложений  
в облаке»**

Онлайн

Разработка

Больше событий в календаре











Ваш аккаунт

Войти  
Регистрация

Разделы

Статьи  
Новости  
Хабы  
Компании  
Авторы  
Песочница

Информация

Устройство сайта  
Для авторов  
Для компаний  
Документы  
Соглашение  
Конфиденциальность

Услуги

Корпоративный блог  
Медийная реклама  
Нативные проекты  
Образовательные программы  
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr